MFC CAS Integration Guide

Overview

This document provides comprehensive instructions for integrating with MFC's Consolidated Account Statement (CAS) service. The integration enables your application to request and retrieve investor data (summary or detailed) through a secure, consent-based flow.

Prerequisites

Before integrating, ensure you have:

1. **Client ID & Client Secret** - For OAuth token generation (provided by MFC team)
2. **Username & Password** - For token generation
3. **Encryption/Decryption Key** - Shared secret key for AES-256 encryption
4. **Private Key** - For JWS signature generation
5. **Public Key** - For signature verification
6. **URL Encryption Key** - 32-byte key for redirect URL encryption
7. **MFC Base URL** - Production or UAT environment URL

Environment Configuration

Store all credentials securely in your environment:

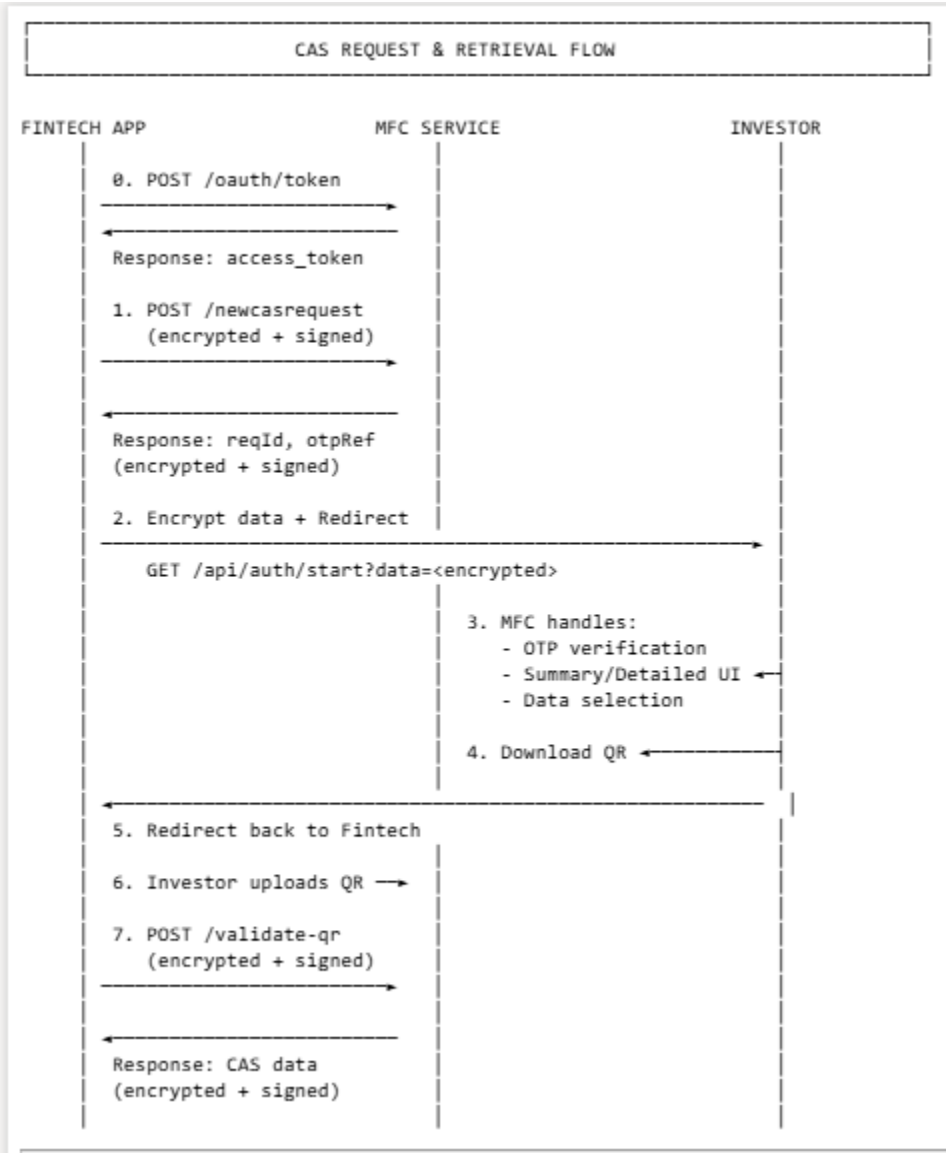
```
# OAuth Credentials
CLIENT_ID="your-client-id"
CLIENT_SECRET="your-client-secret"
USERNAME="your-username"
PASSWORD="your-password"

# Encryption & Signature Keys
ENCRYPTION_KEY="your-shared-encryption-key"
PRIVATE_KEY="your-private-key"
PUBLIC_KEY="your-public-key"
IV="your-initialization-vector"

# URL Encryption (for redirect)
URL_ENCRYPTION_KEY=" odLZvcdVAhq+Re8UstN8u928xRQTE6ym"-32-byte key


# Base URLs
MFC_API_BASE_URL="https://uatservices.mfcentral.com"
MFC_REDIRECT_BASE_URL="https://mfc-cas-uat.mfcentral.com"
```

Integration Flow



Step-by-Step Integration

Step 0: Generate OAuth Token

Before making any API calls, generate an OAuth access token.

Endpoint:

`POST /oauth/token`

Headers:

```
Content-Type: application/x-www-form-urlencoded
Authorization: Basic <Base64(CLIENT_ID:CLIENT_SECRET)>
```

Request Body:

```
username=<USERNAME>
&password=<PASSWORD>
&grant_type=password
```

Response:

```
{
  "access_token": "dd33510a-f161-4818-8d35-65a64f617516",
  "token_type": "bearer",
  "refresh_token": "a0f749fa-2f97-44fd-be93-a5da24a22ca0",
  "expires_in": 2591999,
  "scope": "read write trust"
}
```

Important Notes:

- Token expires in **30 days** - generate a new token before expiry
- Store the `access_token` securely for subsequent API calls
- Use Basic Authentication with Base64-encoded `CLIENT_ID:CLIENT_SECRET`

Example Implementation:

```
async function generateToken() {
  const credentials = Buffer.from(
    `${process.env.CLIENT_ID}:${process.env.CLIENT_SECRET}`
  ).toString('base64');

  const response = await fetch(`${process.env.MFC_API_BASE_URL}/oauth/token
`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/x-www-form-urlencoded',
      'Authorization': `Basic ${credentials}`
    },
    body: new URLSearchParams({
      username: process.env.USERNAME,
      password: process.env.PASSWORD,
      grant_type: 'password'
    })
  });

  const data = await response.json();
```

```
  return data.access_token;
}
```

Step 1: Initiate CAS Request (newcasrequest API)

Call the `newcasrequest` API to generate a request ID and OTP reference. This API requires encryption and signature generation.

Endpoint:

```
POST /api/client/V1/newCasRequest
```

Headers:

```
Content-Type: application/json
ClientId: <YOUR_CLIENT_ID>
Authorization: Bearer <ACCESS_TOKEN>
```

1.1 Prepare Request Payload

```
{
  "clientRefNo": "YOUR_UNIQUE_REF_001",
  "reqId": "",
  "pan": "ABCDE1234F",
  "pekrn": "",
  "email": "",
  "mobile": "+919876543210",
  "fromDate": "31-Mar-2022",
  "toDate": "31-Dec-2024",
  "redirectUrl": "https://your-app.com/cas-callback"
}
```

Field Descriptions:

Field	Type	Required	Description
`clientRefNo`	string	Yes	Your unique internal reference (for tracking)
`reqId`	string	No	Leave empty string (“”) for new requests
`pan`	string	Yes	Investor’s PAN number (10 characters)
`pekrn`	string	No	PEKRN if available, else empty string
`email`	string	Yes*	Investor’s email (*required if mobile not provided)
`mobile`	string	Yes*	Investor’s mobile with country code (*required if email not provided)
`fromDate`	string	Yes	Start date for CAS period (format: DD-MMM-YYYY)
`toDate`	string	Yes	End date for CAS period (format: DD-MMM-YYYY)

Field	Type	Required	Description
`redirectUrl`	string	Yes	URL to redirect investor after QR download

Important Notes:

- `clientRefNo` must be unique across all requests for proper tracking
- Either `email` or `mobile` must be provided (both cannot be provided)
- Mobile number can include country code (e.g., +91 for India)
- Date format must be `DD-MMM-YYYY` (e.g., "31-Mar-2022", "31-Dec-2024")
- `fromDate` and `toDate` define the period for which CAS data will be generated

1.2 Encrypt the Request

Important: This encryption process follows the **standard MFC API encryption flow** (Section 3.3.2 in MFC API documentation). If you're already using other MFC APIs, use the same encryption method.

Use AES-256-CBC encryption with SHA-256 hashed key:

Encryption Specifications:

- **Algorithm:** AES-256-CBC
- **Padding:** PKCS5Padding
- **Key Derivation:** SHA-256 hash of shared key (first 32 bytes)
- **Output:** Base64 encoded

Example Implementation:

```
import crypto from 'crypto';

function encryptRequest(data, encryptionKey, iv) {
  // Take SHA-256 hash of the shared key and use first 32 bytes
  const keyHash = crypto.createHash('sha256').update(encryptionKey).digest(
);
  const key = keyHash.slice(0, 32);

  const cipher = crypto.createCipheriv('aes-256-cbc', key, Buffer.from(iv))
;

  let encrypted = cipher.update(JSON.stringify(data), 'utf8', 'hex');
  encrypted += cipher.final('hex');

  // Return base64 encoded
  return Buffer.from(encrypted, 'hex').toString('base64');
}

// Usage
const iv = '<constant_iv_provided_by_mfc>'; // Use IV provided by MFC
```

```
const encryptedRequest = encryptRequest(requestPayload, process.env.ENCRYPTION_KEY, iv);
```

1.3 Generate Signature (JWS)

Important: This signature generation follows the **standard MFC API signature flow** (Section 3.3.3 in MFC API documentation). Use the same method as your existing MFC API integrations.

Generate a JSON Web Signature (JWS) for the encrypted request:

Reference: Use JWS library to sign the encrypted payload

Example Implementation:

```
import jwt from 'jsonwebtoken';

function generateSignature(encryptedText, privateKey) {
  // Generate JWS signature using private key
  const signature = jwt.sign(
    { data: encryptedText },
    privateKey,
    { algorithm: 'RS256' }
  );

  return signature;
}

// Usage
const signature = generateSignature(encryptedRequest, process.env.PRIVATE_KEY);
```

1.4 Submit Request

Request Format:

```
{
  "signature": "<JWS_SIGNATURE>",
  "request": "<ENCRYPTED_REQUEST>"
}
```

Complete Example:

```
async function callNewCASRequest(pan, email, mobile, clientRefNo, fromDate, toDate, redirectUrl) {
  // Step 1: Prepare payload
```

```
  const payload = {
    clientRefNo,
    reqId: "",
    pan,
    pekrn: "",
    email: email || "",
    mobile: mobile || "",
    fromDate,
    toDate,
    redirectUrl
  };

  // Step 2: Encrypt
  const encryptedRequest = encryptRequest(
    payload,
    process.env.ENCRYPTION_KEY,
    process.env.IV
  );

  // Step 3: Generate signature
  const signature = generateSignature(encryptedRequest, process.env.PRIVATE
_KEY);

  // Step 4: Submit request
  const response = await fetch(
    `${process.env.MFC_API_BASE_URL}/api/client/V1/newCasRequest`,
    {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'ClientId': process.env.CLIENT_ID,
        'Authorization': `Bearer ${accessToken}`
      },
      body: JSON.stringify({
        signature,
        request: encryptedRequest
      })
    }
  );

  return await response.json();
}
```

1.5 Process Response

Response Format:

```
{
  "signature": "<JWS_SIGNATURE>",
  "response": "<ENCRYPTED_RESPONSE>"
}
```

Important: Response processing follows the **standard MFC API flow** (Sections 3.3.5 and 3.3.6 in MFC API documentation):

1. Verify signature using public key
2. Decode base64
3. Decrypt using AES-256-CBC
4. Parse JSON

Steps to Process:

1. **Verify Signature** - Validate JWS signature using public key (Section 3.3.5)
2. **Decrypt Response** - Decrypt using AES-256-CBC (Section 3.3.6)
3. **Extract Data** - Parse JSON to get `reqId` and `otpRef`

Example Implementation:

```
function verifySignature(signature, publicKey) {
  try {
    const decoded = jwt.verify(signature, publicKey, { algorithms: ['RS256'
] });
    return decoded;
  } catch (error) {
    throw new Error('Signature verification failed');
  }
}

function decryptResponse(encryptedText, encryptionKey, iv) {
  // Decode base64 first
  const encryptedBuffer = Buffer.from(encryptedText, 'base64');

  // Take SHA-256 hash of the shared key and use first 32 bytes
  const keyHash = crypto.createHash('sha256').update(encryptionKey).digest(
);
  const key = keyHash.slice(0, 32);

  const decipher = crypto.createDecipheriv('aes-256-cbc', key, Buffer.from(
iv));

  let decrypted = decipher.update(encryptedBuffer);
  decrypted = Buffer.concat([decrypted, decipher.final()]);
```

```
  return JSON.parse(decrypted.toString('utf8'));
}

// Process response
async function processNewCASResponse(responseData) {
  // Verify signature
  verifySignature(responseData.signature, process.env.PUBLIC_KEY);

  // Decrypt response
  const decryptedData = decryptResponse(
    responseData.response,
    process.env.ENCRYPTION_KEY,
    process.env.IV
  );

  return decryptedData; // Contains: { reqId, otpRef, ... }
}
```

Decrypted Response:

```
{
  "reqId": 3102549,
  "otpRef": "2d707c52-50a400601892a2-9c0eb08961c4",
  "userSubjectReference": "",
  "clientRefNo": "newCasQraCo_a19904"
}
```

Response Fields:

Field	Type	Description
`reqId`	number	MFC-generated request ID for tracking
`otpRef`	string	OTP reference for investor consent verification
`userSubjectReference`	string	User subject reference (if applicable)
`clientRefNo`	string	Your original client reference number

Important: Store the `reqId` and `clientRefNo` securely - you'll need them for the validateQRCode API call.

Step 2: Encrypt Data and Redirect Investor

Using the `reqId` and `otpRef` from Step 1, encrypt the investor data for URL redirection.

Important: This encryption is **different** from API encryption. Use simple AES-256-CBC with the URL encryption key.

Encryption Specification:

- **Algorithm:** AES-256-CBC
- **Key:** 32-byte URL encryption key (not hashed)
- **IV:** Random 16-byte initialization vector
- **Format:** `<IV_HEX>:<ENCRYPTED_DATA_HEX>`

Payload Field Specifications:

Field	Type	Required	Description
`reqId`	string	Yes	Unique request ID from step1
`otpRef`	string	Yes	OTP reference from step1
`token`	string	No	Leave as Blank
`clientRefNo`	string	Yes	Your unique internal reference (for tracking)
`pan`	string	Yes	User's PAN
`pekrn`	string	No	PEKRN if available, else empty string
`email`	string	Yes*	Investor's email (*required if mobile not provided)
`mobile`	string	Yes*	Investor's mobile with country code (*required if email not provided)
`redirectUrl`	string	Yes	URL to redirect investor after QR download

Example Implementation:

```
function encryptForRedirect(reqId, otpRef, pan, email, mobile, clientRefNo,
redirectUrl) {
  const SECRET_KEY = process.env.URL_ENCRYPTION_KEY; // 32 bytes, not hashe
d
  const ALGORITHM = 'aes-256-cbc';

  // Prepare payload
  const dataToEncrypt = {
    reqId,
    otpRef,
    token,
    pan,
    email,
    mobile,
    clientRefNo,
    redirectUrl
  };

  // Generate random IV (16 bytes)
  const iv = crypto.randomBytes(16);
```

```
  // Create cipher with URL encryption key (not hashed)
  const cipher = crypto.createCipheriv(
    ALGORITHM,
    Buffer.from(SECRET_KEY), // Use key directly, no hashing
    iv
  );

  // Encrypt
  let encrypted = cipher.update(JSON.stringify(dataToEncrypt), 'utf8', 'hex
');
  encrypted += cipher.final('hex');

  // Combine IV and encrypted data
  const combinedData = iv.toString('hex') + ':' + encrypted;

  // URL encode
  const encodedData = encodeURIComponent(combinedData);

  // Construct redirect URL
  return `${process.env.MFC_BASE_URL}/api/auth/start?data=${encodedData}`;
}

// Usage
const redirectUrl = encryptForRedirect(
  reqId,
  otpRef,
  "ABCDE1234F",
  "investor@example.com",
  "+919876543210",
  "YOUR_UNIQUE_REF_001",
  "https://your-app.com/cas-callback"
);

// Redirect investor
res.redirect(redirectUrl); // Express.js
```

Important Notes:

- If you want investors to select **detailed data** instead of summary, instruct them to choose the "Detailed" option in the MFC UI
- All OTP verification and data selection is handled by MFC's frontend
- You do not need to call any additional APIs during this step

Step 3: MFC Handles Investor Interaction

Once redirected to MFC, the following happens automatically:

1. **OTP Verification** - Investor receives and enters OTP for consent
2. **Data Type Selection** - Investor chooses between:
   - **Summary CAS** - Basic consolidated statement
   - **Detailed CAS** - Comprehensive transaction details
3. **QR Code Generation** - MFC generates a QR code containing the requested data
4. **Download** - Investor can:
   - Download the QR code directly

No action required from your application during this step.

Step 4: QR Download & Redirection

After the investor downloads the QR code, they are automatically redirected back to your application using the `redirectUrl` provided in Step 2.

Terminal State: QR code download completes the MFC flow.

Step 5: Investor Uploads QR Code

The investor returns to your application and uploads the downloaded QR code image.

Frontend Implementation Example (React):

```
import React, { useState } from "react";

export default function QRUploadForm({ reqId, clientRefNo }) {
  const [isLoading, setIsLoading] = useState(false);
  const [result, setResult] = useState(null);

  const handleFileChange = async (e) => {
    const file = e.target.files?.[0];
    if (!file) return;

    setIsLoading(true);

    try {
      // Convert to base64
      const base64 = await new Promise((resolve, reject) => {
        const reader = new FileReader();
        reader.onload = () => {
          const dataUrl = reader.result;
          // Remove data URL prefix (e.g., "")
```

```
          const base64String = dataUrl.split(",")[1];
          resolve(base64String);
        };
        reader.onerror = reject;
        reader.readAsDataURL(file);
      });

      // Call your backend to validate QR
      const response = await fetch("/api/cas/validate-qr", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ reqId, clientRefNo, qrCode: base64 })
      });

      if (!response.ok) {
        const error = await response.json();
        throw new Error(error.message || "QR validation failed");
      }

      const data = await response.json();
      setResult(data);
      alert("CAS data retrieved successfully!");
    } catch (error) {
      alert(`Error: ${error.message}`);
    } finally {
      setIsLoading(false);
    }
  };

  return (
    <div>
      <h3>Upload Your QR Code</h3>
      <input
        type="file"
        accept="image/png,image/jpeg"
        onChange={handleFileChange}
        disabled={isLoading}
      />
      {isLoading && <p>Validating QR code...</p>}
      {result && <pre>{JSON.stringify(result, null, 2)}</pre>}
    </div>
  );
}
```

Note: The backend endpoint `/api/cas/validateQRCode` should handle the encryption, signature generation, API call to MFC, and decryption as shown in Step 6.

Step 6: Validate QR Code

Call the `validate-qr` API to extract and retrieve the CAS data.

Important: This API uses the **same encryption, signature, and decryption process** as the `newcasrequest` API (standard MFC API flow from Section 3.3 of MFC API documentation).

Endpoint:

```
POST /api/client/V1/validateQRCode
```

Headers:

```
Content-Type: application/json
ClientId: <YOUR_CLIENT_ID>
Authorization: Bearer <ACCESS_TOKEN>
```

6.1 Prepare Request Payload

```
{
  "reqId": "3087591",
  "clientRefNo": "YOUR_UNIQUE_REF_001",
  "qrCode": "iVBORw0KGgoAAAANSUhEUgAA..."
}
```

Field Descriptions:

Field	Type	Required	Description
`reqId`	string	Yes	Request ID from Step 1
`clientRefNo`	string	Yes	Your internal reference number
`qrCode`	string	Yes	Base64-encoded QR code image

6.2 Complete Implementation

```
async function validateQRCode(reqId, clientRefNo, qrCodeBase64) {
  // Step 1: Prepare payload
  const payload = {
    reqId,
    clientRefNo,
    qrCode: qrCodeBase64
  };

  // Step 2: Encrypt (same method as newCasRequest)
  const encryptedRequest = encryptRequest(
    payload,
    process.env.ENCRYPTION_KEY,
```

```
    process.env.IV
  );

  // Step 3: Generate signature
  const signature = generateSignature(encryptedRequest, process.env.PRIVATE
_KEY);

  // Step 4: Submit request
  const response = await fetch(
    `${process.env.MFC_API_BASE_URL}/api/client/V1/validateQRCode`,
    {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'ClientId': process.env.CLIENT_ID,
        'Authorization': `Bearer ${accessToken}`
      },
      body: JSON.stringify({
        signature,
        request: encryptedRequest
      })
    }
  );

  const responseData = await response.json();

  // Step 5: Verify signature
  verifySignature(responseData.signature, process.env.PUBLIC_KEY);

  // Step 6: Decrypt response
  const decryptedData = decryptResponse(
    responseData.response,
    process.env.ENCRYPTION_KEY,
    process.env.IV
  );

  return decryptedData;
}
```

6.3 Response Format

Response (Summary):

```
{
    "reqId": "3122359",
```

```
    "pan": "AGVPN4690G",
    "mobile": "9550755111",
    "email": "",
    "pekrn": "",
    "data": [
        {
            "summary": [
                {
                    "amc": "108",
                    "amcName": "UTI MUTUAL FUND",
                    "currentMktValue": 16942.16,
                    "costValue": 14999.25,
                    "gainLoss": 1942.91,
                    "gainLossPercentage": 13,
                    "isDemat": "N"
                }
            ],
            "schemes": [
                {
                    "amc": "108",
                    "amcName": "UTI MUTUAL FUND",
                    "folio": "599366447236",
                    "investorName": "John Doe",
                    "age": 39,
                    "mobile": "9550755111",
                    "email": "nutalapatiharsha@gmail.com",
                    "taxStatus": "01",
                    "modeOfHolding": "SINGLE",
                    "transactionSource": "",
                    "schemeCode": "EQGP",
                    "schemeName": "UTI Flexi Cap Fund - Regular Plan",
                    "schemeOption": "GROWTH",
                    "idcwChangeAllowed": "False",
                    "assetType": "EQUITY",
                    "schemeType": "EQUITY FUND",
                    "nav": 321.7826,
                    "navDate": "01-Jan-2026",
                    "closingBalance": 16.521,
                    "availableUnits": 16.521,
                    "lienEligibleUnits": 16.521,
                    "availableAmount": 5316.17,
                    "currentMktValue": 5316.17,
                    "costValue": 4999.75,
                    "gainLoss": 316.42,
                    "gainLossPercentage": 6.3,
                    "lienUnitsFlag": "N",
                    "isin": "INF789F01513",
```

```
            "brokerCode": "ARN-154065",
            "brokerName": "Gajwinkar Srikanth",
            "decimalUnits": 3,
            "decimalAmount": 2,
            "decimalNav": 4,
            "isDemat": "N",
            "purAllow": "Y",
            "redAllow": "Y",
            "swtAllow": "Y",
            "sipAllow": "Y",
            "stpAllow": "Y",
            "swpAllow": "Y",
            "planMode": "REGULAR",
            "dpId": "",
            "rtaName": "KFIN",
            "nomineeStatus": "Y",
            "kycStatus": "02",
            "bank": {
                "accountNo": "068001510540",
                "accountType": "SAVING",
                "name": "ICICI BANK LIMITED",
                "branch": "",
                "city": "HYDERABAD",
                "pincode": "",
                "micr": "",
                "ifsc": "ICIC0000680",
                "neftIfsc": "ICIC0000680"
            }
        },
        {
            "amc": "108",
            "amcName": "UTI MUTUAL FUND",
            "folio": "599366447236",
            "investorName": " John Doe",
            "age": 39,
            "mobile": "9550766141",
            "email": " JohnDoe@gmail.com",
            "taxStatus": "01",
            "modeOfHolding": "SINGLE",
            "transactionSource": "",
            "schemeCode": "ESGP",
            "schemeName": "UTI Equity Savings Fund - Regular Plan",
            "schemeOption": "GROWTH",
            "idcwChangeAllowed": "False",
            "assetType": "EQUITY",
            "schemeType": "EQUITY FUND",
            "nav": 19.038,
```

```
            "navDate": "01-Jan-2026",
            "closingBalance": 291.198,
            "availableUnits": 291.198,
            "lienEligibleUnits": 291.198,
            "availableAmount": 5543.83,
            "currentMktValue": 5543.83,
            "costValue": 4999.75,
            "gainLoss": 544.08,
            "gainLossPercentage": 10.9,
            "lienUnitsFlag": "N",
            "isin": "INF789F1A744",
            "brokerCode": "ARN-154065",
            "brokerName": "Gajwinkar Srikanth",
            "decimalUnits": 3,
            "decimalAmount": 2,
            "decimalNav": 4,
            "isDemat": "N",
            "purAllow": "Y",
            "redAllow": "Y",
            "swtAllow": "Y",
            "sipAllow": "Y",
            "stpAllow": "Y",
            "swpAllow": "Y",
            "planMode": "REGULAR",
            "dpId": "",
            "rtaName": "KFIN",
            "nomineeStatus": "Y",
            "kycStatus": "02",
            "bank": {
                "accountNo": "068001510540",
                "accountType": "SAVING",
                "name": "ICICI BANK LIMITED",
                "branch": "",
                "city": "HYDERABAD",
                "pincode": "",
                "micr": "",
                "ifsc": "ICIC0000680",
                "neftIfsc": "ICIC0000680"
            }
        },
        {
            "amc": "108",
            "amcName": "UTI MUTUAL FUND",
            "folio": "599366447236",
            "investorName": "HARSHA VARDHAN NUTALAPATI",
            "age": 39,
            "mobile": "9560755171",
```

```
"email": " JohnDoe@gmail.com",
"taxStatus": "01",
"modeOfHolding": "SINGLE",
"transactionSource": "",
"schemeCode": "GPGP",
"schemeName": "UTI Healthcare Fund - Regular Plan",
"schemeOption": "GROWTH",
"idcwChangeAllowed": "False",
"assetType": "EQUITY",
"schemeType": "EQUITY FUND",
"nav": 284.6919,
"navDate": "01-Jan-2026",
"closingBalance": 21.364,
"availableUnits": 21.364,
"lienEligibleUnits": 21.364,
"availableAmount": 6082.16,
"currentMktValue": 6082.16,
"costValue": 4999.75,
"gainLoss": 1082.41,
"gainLossPercentage": 21.6,
"lienUnitsFlag": "N",
"isin": "INF789F01695",
"brokerCode": "ARN-154065",
"brokerName": "Gajwinkar Srikanth",
"decimalUnits": 3,
"decimalAmount": 2,
"decimalNav": 4,
"isDemat": "N",
"purAllow": "Y",
"redAllow": "Y",
"swtAllow": "Y",
"sipAllow": "Y",
"stpAllow": "Y",
"swpAllow": "Y",
"planMode": "REGULAR",
"dpId": "",
"rtaName": "KFIN",
"nomineeStatus": "Y",
"kycStatus": "02",
"bank": {
    "accountNo": "068001510540",
    "accountType": "SAVING",
    "name": "ICICI BANK LIMITED",
    "branch": "",
    "city": "HYDERABAD",
    "pincode": "",
    "micr": "",
```

```
                        "ifsc": "ICIC0000680",
                        "neftIfsc": "ICIC0000680"
                    }
                }
            ]
        }
    ],
    "errorMessage": "",
    "portfolio": [
        {
            "currentMktValue": "16942.16",
            "costValue": "14999.25",
            "gainLoss": "1942.91",
            "gainLossPercentage": "12.95",
            "isDemat": "N"
        },
        {
            "currentMktValue": "0.00",
            "costValue": "0.00",
            "gainLoss": "0.00",
            "gainLossPercentage": "0.00",
            "isDemat": "Y"
        }
    ],
    "investorDetails": {},
    "statementHoldingFilter": null
}
```


Response (Detailed):


```
{
    "reqId": "2638435-590807058",
    "pan": "CFRPP0494R",
    "pekrn": "",
    "email": "",
    "fromDate": "01-Apr-1980",
    "toDate": "29-Aug-2024",
    "data": [
        {
            "dtTransaction": [
                {
                    "email": "",
                    "amc": "H",
                    "amcName": "MFHMF Test Fund",
                    "folio": "17064219",
                    "checkDigit": "63",
```

```
                "trxnDate": "24-SEP-2020",
                "postedDate": "24-SEP-2020",
                "scheme": "02",
                "schemeName": "HDFC Flexi Cap Fund - Regular Plan - Growth ",
                "trxnDesc": "Address Updated from KRA Data",
                "trxnAmount": "0",
                "trxnUnits": "0",
                "purchasePrice": "0",
                "sttTax": "0",
                "tax": "0",
                "totalTax": "0",
                "trxnMode": "N",
                "stampDuty": "0",
                "trxnCharge": "0",
                "trxnTypeFlag": "",
                "isin": "INF179K01608"
            },
            {
                "email": "",
                "amc": "H",
                "amcName": "MFHMF Test Fund",
                "folio": "17064219",
                "checkDigit": "63",
                "trxnDate": "28-SEP-2020",
                "postedDate": "28-SEP-2020",
                "scheme": "02",
                "schemeName": "HDFC Flexi Cap Fund - Regular Plan - Growth ",
                "trxnDesc": "Change Of IDCW  Option - Systematic",
                "trxnAmount": "0",
                "trxnUnits": "0",
                "purchasePrice": "0",
                "sttTax": "0",
                "tax": "0",
                "totalTax": "0",
                "trxnMode": "N",
                "stampDuty": "0",
                "trxnCharge": "0",
                "trxnTypeFlag": "",
                "isin": "INF179K01608"
            }
        ],
        "dtSummary": [
            {
                "email": "",
                "amc": "H",
                "amcName": "MFHMF Test Fund",
                "folio": "17064219",
```

```
                    "scheme": "02",
                    "schemeName": "HDFC Flexi Cap Fund - Regular Plan - Growth ",
                    "kycStatus": "1",
                    "brokerCode": "ARN-48944",
                    "brokerName": "Mohanlal Jangir",
                    "rtaCode": "CAMS",
                    "decimalUnits": "3",
                    "decimalAmount": "2",
                    "decimalNav": "3",
                    "lastTrxnDate": "05-MAY-2022",
                    "openingBal": "0.000",
                    "marketValue": "6542.10",
                    "nav": "150.000",
                    "closingBalance": "43.614",
                    "lastNavDate": "11-JUL-2024",
                    "isDemat": "N",
                    "assetType": "EQUITY",
                    "isin": "INF179K01608",
                    "nomineeStatus": "N",
                    "taxStatus": "01",
                    "costValue": "38000"
                },
                {
                    "email": "",
                    "amc": "H",
                    "amcName": "MFHMF Test Fund",
                    "folio": "17064219",
                    "scheme": "54",
                    "schemeName": "HDFC Low Duration Fund - Regular Plan - Growth
(formerly HDFC Cash
Management Fund - Treasury Advantage Plan)",
                    "kycStatus": "1",
                    "brokerCode": "ARN-48944",
                    "brokerName": "Mohanlal Jangir",
                    "rtaCode": "CAMS",
                    "decimalUnits": "3",
                    "decimalAmount": "2",
                    "decimalNav": "4",
                    "lastTrxnDate": "04-FEB-2021",
                    "openingBal": "0.000",
                    "marketValue": "133924.00",
                    "nav": "100.0000",
                    "closingBalance": "1339.240",
                    "lastNavDate": "12-AUG-2024",
                    "isDemat": "N",
                    "assetType": "DEBT",
                    "isin": "INF179K01442",
```

```
                    "nomineeStatus": "N",
                    "taxStatus": "01",
                    "costValue": "60000"
                }
            ]
        }
    ],
    "investorDetails": {
        "address": {
            "address1": "Address1 of Folio: 15711754",
            "address2": "Address2 of Folio: 15711754",
            "address3": "Address3 of Folio: 15711754",
            "city": "HOWRAH",
            "district": "",
            "state": "West Bengal",
            "pincode": "711204",
            "country": "India"
        },
        "mobile": "+917874806606",
        "investorFirstName": "JQLVAAIJNJSI",
        "investorMiddleName": "",
        "investorLastName": "",
        "email": "dev.sandipk@gmail.com"
    }
}
```

Security & Encryption Summary

Two Different Encryption Methods

Your integration uses **TWO different encryption methods** for different purposes:

1. API Request/Response Encryption (Steps 1 & 6)

Used for: `newcasrequest` and `validateQRCode` API calls

Important Note: The encryption, decryption, and signature generation process for these API calls follows the **same existing MFC API flow** used in other MFC integrations (as documented in section 3.3 of the MFC API documentation). If you're already integrated with other MFC APIs, you can use the same encryption/decryption methods.

Specifications:

- **Algorithm:** AES-256-CBC
- **Key:** SHA-256 hash of shared encryption key (first 32 bytes)
- **IV:** Constant IV provided by MFC
- **Padding:** PKCS5Padding
- **Output:** Base64 encoded
- **Additional:** JWS signature required

```
// Key derivation
const keyHash = crypto.createHash('sha256').update(encryptionKey).digest();
const key = keyHash.slice(0, 32);
```

Reference: This follows the standard MFC API security flow documented in the MFC API Integration Guide (Section 3.3 - Submitting Request to MFC).

2. URL Redirect Encryption (Step 2)

Used for: Encrypting data in redirect URL parameter

Specifications:

- **Algorithm:** AES-256-CBC
- **Key:** 32-byte URL encryption key (used directly, not hashed)
- **IV:** Random 16-byte IV (generated per request)
- **Format:** `<IV_HEX>:<ENCRYPTED_DATA_HEX>`
- **Output:** URL-encoded string
- **Additional:** No signature required

```
// Use key directly without hashing
const cipher = crypto.createCipheriv(ALGORITHM, Buffer.from(SECRET_KEY), iv);
```

ERROR:

A failure response is preceded by the corresponding 40x or 50 x HTTP headers. The code key in the response contains the value error. The message key contains a textual description of the error.

Below are the possible error codes:

Status Code	Actual Status	Reason/Description
400	Valid Business error	In this case, refer error description of the error & take action, these are generally business validation errors. Basis the error received, you can ask the user to remediate certain fields & resubmit the transaction again
500	Internal server error or Connectivity error	In this case, refer error description of the error & take action, these are generally technical errors.
401	Unauthorized	Invalid token or client Id
422	Un-processable Entity (WebDAV)	Invalid Input or Request Format
404	Not Found	The requested resource could not be found on the server. Please verify the endpoint URL and try again.
202	Accepted	In this case, the request has been accepted for processing, but the processing has not been completed.
200	Successful	Request has been accepted and it is a successful request

---

For Front End reference:

Demo Page: MFC CAS Flow

UAT OTP format:

00 + last 4 digits of the PAN

Example:

PAN: ABCDE1234F

OTP: 001234

UAT Test Data:

EMAIL	test@kfintech.com
MOBILE	9239874560

List of PANs

AQQPS7576R
AATPJ9485B
ANKPJ4391C
AAQPH6449C
AOBPC3943B
AAAPS6436J
AQQPS7576R
DZMPA3911G
BVPPB8007F